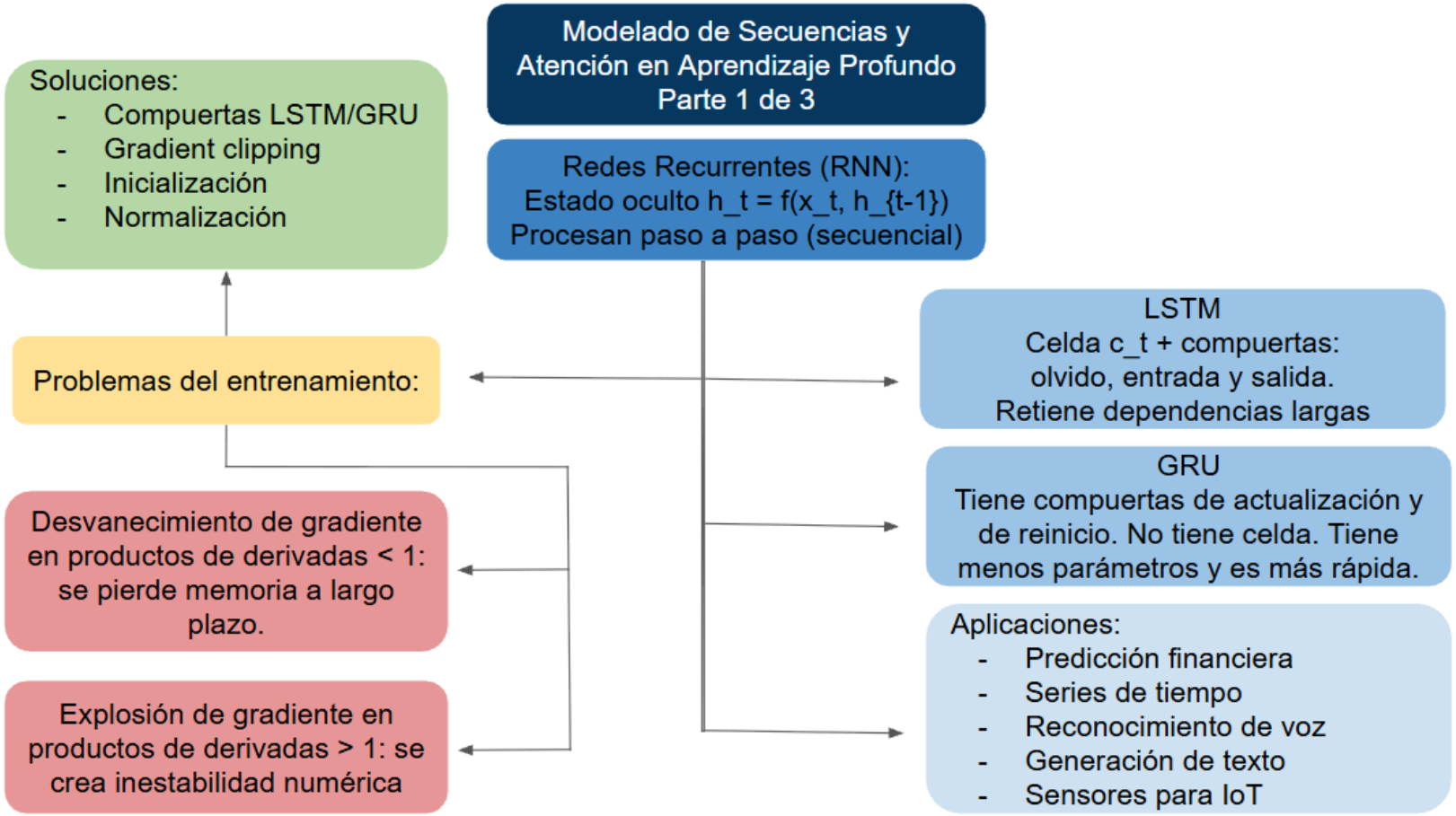
## 1) Introduccion:

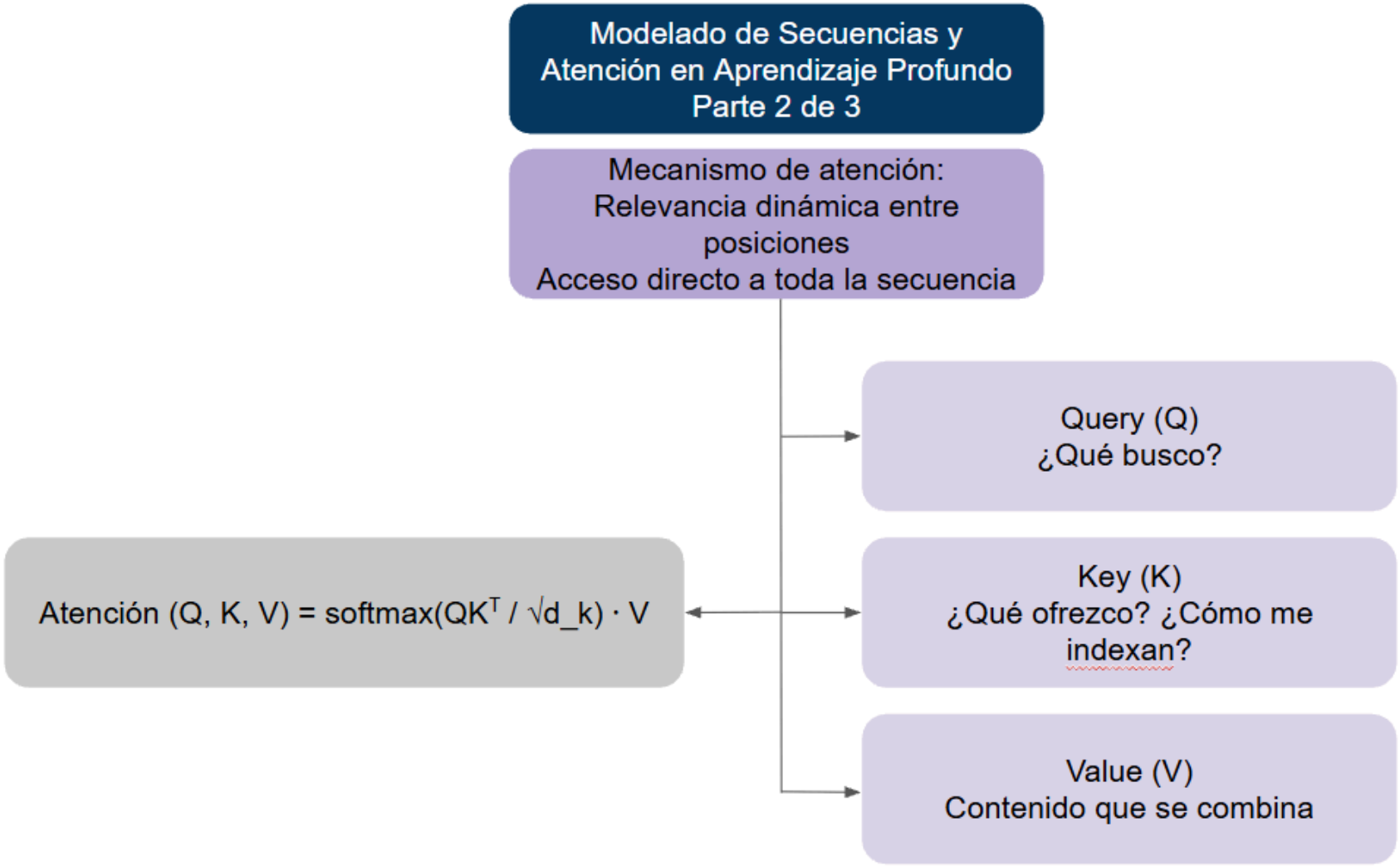
La gran mayoría de los datos en el mundo real son secuenciales (texto, voz, señales de sensores). A diferencia de una imagen o datos tabulares, en los datos secuenciales el orden importa y cada elemento depende del contexto que lo precede. Las redes convolucionales (CNN, por sus siglas en inglés) explotan la localidad espacial y las redes generativas adversarias (GAN) aprenden a sintetizar datos mediante la competencia entre un generador y un discriminador. Para las secuencias, los modelos necesarios son las redes recurrentes y los mecanismos de atención.

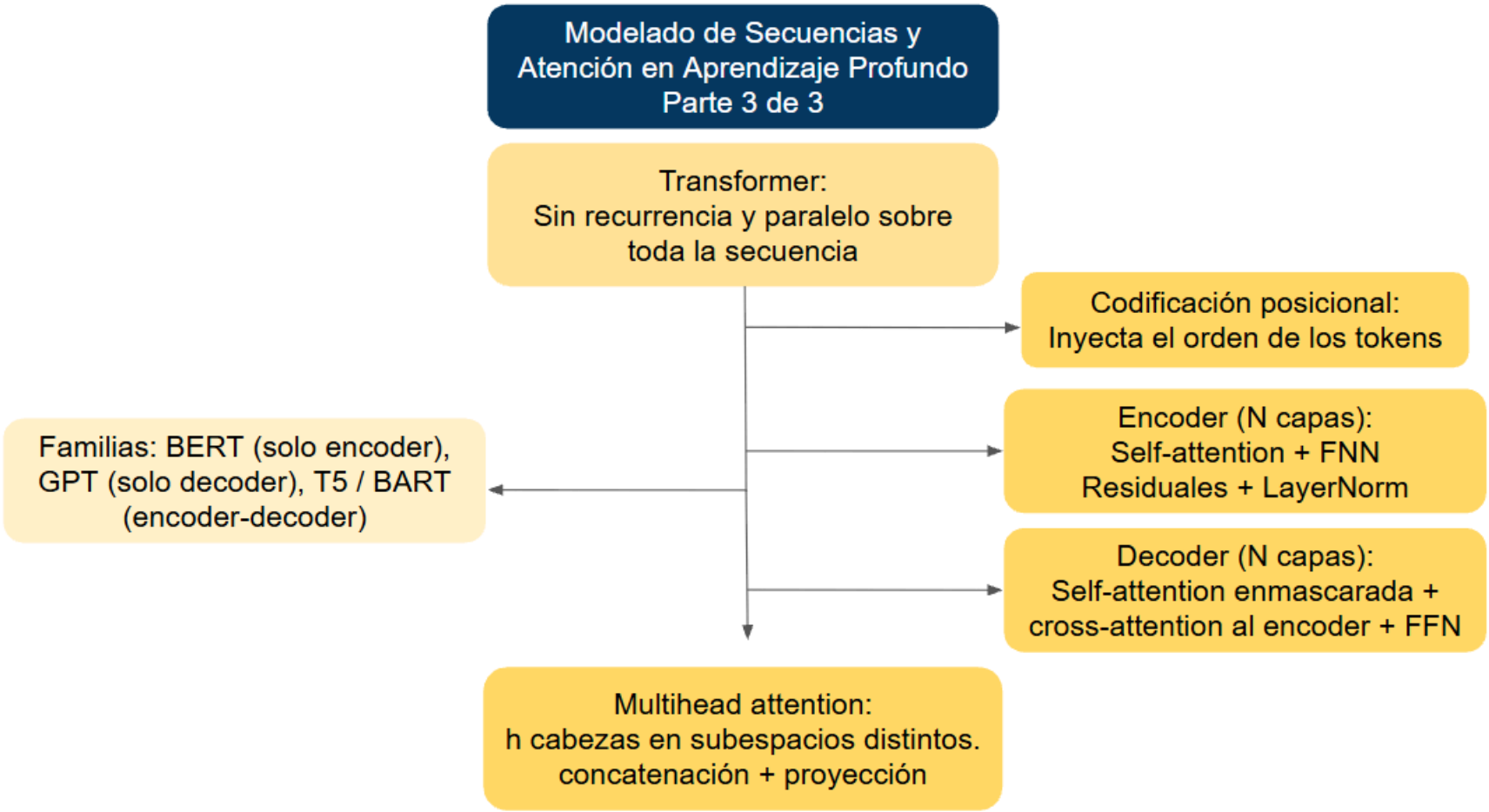
Este informe contiene tres partes: 1) organizar los conceptos esenciales de RNN, LSTM, GRU, mecanismos de atención y Transformer en un mapa conceptual comparativo 2) análisis crítico de dos casos reales. Un caso es un modelo hibrido GRU-LSTM en reconocimiento de voz y el otro es con Vision Transformers en visión por computadora 3) implementación comparativa de un LSTM de generación de texto entrenado desde cero y un Transformer preentrenado (T5-small) para resumen automático con la librería Hugging Face. El código completo se entrega en un notebook de Google Colab.

## 2) Mapa Conceptual:

Los siguientes mapas conceptuales resumen las relaciones entre las arquitecturas. El mapa conceptual fue dividido en tres partes para tener una visualización más clara. La rama de redes recurrentes parte de la RNN simple. En clase aprendimos que hay dos varones con compuertas: LSTM y GRU. Además, está relacionada con los problemas de entrenamiento (desvanecimiento y explosión de gradiente) y sus soluciones. La rama de mecanismos de atención define Query, Key y Value. Finalmente, la rama de Transformer muestra que la atención multi-cabeza es el bloque central del encoder y del decoder.




Modelado de Secuencias y Atención en Aprendizaje Profundo Parte 2 de 3
Mecanismo de atención: Relevancia dinámica entre posiciones Acceso directo a toda la secuencia
Query (Q) ¿Qué busco?
Key (K) ¿Qué ofrezco? ¿Cómo me indexan?
Value (V) Contenido que se combina
Atención (Q, K, V) = softmax(QK^T / √d_k) · V


Modelado de Secuencias y Atención en Aprendizaje Profundo Parte 3 de 3
Transformer: Sin recurrencia y paralelo sobre toda la secuencia
Codificación posicional: Inyecta el orden de los tokens
Encoder (N capas): Self-attention + FNN Residuales + LayerNorm
Decoder (N capas): Self-attention enmascarada + cross-attention al encoder + FFN
Familias: BERT (solo encoder), GPT (solo decoder), T5 / BART (encoder-decoder)
Multihead attention: h cabezas en subespacios distintos. concatenación + proyección

***Redes Recurrentes (RNN)***
Una RNN procesa la secuencia de un elemento a la vez. Además, mantiene un estado oculto que sirve para resumir lo visto hasta el momento. En cada paso combina la entrada actual con el estado anterior usando los mismos pesos. Estas redes se entrenan con retropropagación a través del tiempo. Son adecuadas para secuencias de longitud variable, pero la naturaleza secuencial impide paralelizar el cálculo a lo largo del tiempo.

***LSTM***
La variable de *Long Short-Term Memory* añade una celda de memoria c_t con un camino de actualización aditivo, controlada por tres compuertas con activación sigmoide (valores entre 0 y 1). En este tipo de redes, la compuerta de olvido decide la información que se descarta de la memoria, la compuerta de entrada decide la información nueva que se escribe y la compuerta de salida decide la parte de la memoria que se expone como estado oculto. Sus aplicaciones son el modelado de lenguaje, reconocimiento de voz, traducción (antes de los Transformers) y pronóstico de series temporales.

***GRU***
La variable de *Gated Recurrent Unit* simplifica la LSTM: elimina la celda separada y usa dos compuertas. Una de sus compuertas es la de actualización, usada para saber cuánto del estado anterior se conserva. La otra compuerta es la de reinicio, usada para saber cuánto del pasado se usa para calcular el candidato. Tiene menos parámetros y suele entrenar más rápido con un rendimiento comparable en muchas tareas.

***Desvanecimiento y xeplosion del Gradiente***
En la retropropagación a través del tiempo, el gradiente de la pérdida respecto a un estado lejano en el pasado contiene un producto de muchos jacobianos (uno por paso de tiempo). Si los factores tienen una norma menor que 1, el producto decae exponencialmente y se genera el desvanecimiento del gradiente. Como resultado, la red no aprende dependencias largas. En el caso opuesto que los factores tienen una norma mayor que 1, entonces el producto crece exponencialmente y se genera la explosión de gradiente. Como resultado, las pérdidas son inestables.

- **Compuertas LSTM/GRU**: el camino aditivo de la celda permite que el gradiente fluya con menor atenuación, lo que mitiga el desvanecimiento.
- **Recorte de gradiente (gradient clipping)**: limita la norma del gradiente y controla la explosión. Lo utilizo en mi código, la norma máxima es de 1.0.
- **Inicialización y normalización adecuadas**: acortan el camino entre posiciones distantes a un solo paso.

***Mecanismo de atención: Query, Key y Value***
Estos mecanismos surgieron para que un decodificador pudiera ver selectivamente distintas partes de la entrada en lugar de depender de un único vector resumen. Cada elemento de la secuencia genera tres vectores mediante proyecciones lineales aprendidas:
- **Query (Q)**: representa lo que el elemento actual está “buscando”.

- **Key (K)**: representa lo que cada elemento “ofrece” para ser encontrado.
- **Value (V)**: el contenido que se transmite si el elemento resulta relevante.

La relevancia se calcula como la similitud, usando el producto punto, entre cada Query y todas las Keys, después se escala por √d_k para estabilizar los gradientes, se normaliza con la función de activación softmax y se usa como peso para promediar los Values:

$$Atención(Q, K, V) = softmax( Q \cdot K^T / \sqrt{d_k} ) \cdot V$$

De esta forma, la relevancia se asigna dinámicamente: depende del contenido de la secuencia y no de una distancia fija. Cada posición accede directamente a cualquier otra, lo que reduce la distancia entre dependencias a un solo paso.

***Arquitectura Transformer***

El Transformer elimina la recurrencia. Se basa solamente en atención y capas densas. Sus componentes son los siguientes:

- **Embeddings + codificación posicional**: como la atención no conoce el orden, se suma a cada embedding una señal de posición.
- **Encoder (N capas)**: cada capa contiene self-attention multi-cabeza y una red feed-forward por posición con conexiones residuales y normalización de capa. Produce una representación contextual de la entrada.
- **Decoder (N capas)**: self-attention enmascarada (impide ver token futuros), atención cruzada (cross-attention) donde las Queries vienen del decoder y las Keys/Values del encoder y una red feed-forward.
- **Multi-head attention**: se ejecutan h atenciones en paralelo sobre subespacios distintos (el modelo original usa h=8, d_model= 512 y 6 capas). Los resultados se concatenan y proyectan. Cada cabeza puede especializarse en un tipo de relación (sintáctica, posicional y de largo alcance).

Como todas las posiciones se procesan simultáneamente durante el entrenamiento, el Transformer aprovecha el paralelismo de los GPUs. El costo es que el self-attention escala de forma cuadrática con la longitud de la secuencia, $O(n^2 \cdot d)$. En contraste, las operaciones de secuencia RNN escalan de forma lineal $O(n)$. Las variantes más usadas son BERT (solamente encoder), GPT (solamente decoder) y T5 / BART (encoder-decoder).

***Tabla Comparativa - Comparación de arquitecturas de secuencia:***

| Criterio | RNN Simple | LSTM | GRU | Transformer |
|---|---|---|---|---|
| Procesamiento | Secuencial | Secuencial | Secuencial | Paralelo (toda la secuencia) |
| Memoria de largo plazo | Débil (gradiente se desvanece) | Buena (celda + compuertas) | Buena (2 compuertas) | Directa vía atención |
| Parámetros por unidad | Pocos | Más (3 compuertas + celda) | Intermedio | Muchos (escala a miles de millones) |
| Costo con la longitud n | O(n) secuencial | O(n) secuencial | O(n) secuencial | $O(n^2)$ secuencial |
| Datos requeridos | Pocos | Moderados | Moderados | Muchos (se mitiga con preentrenamiento) |
| Usos típicos | Secuencias cortas | Series de tiempo, voz, texto | Igual que LSTM con menos cómputo | Traducción, resumen, generación, visión |

***Otras arquitecturas comunes: CNN y GAN***

Para poder comprender bien las RNN y Transformers dentro del panorama del aprendizaje profundo conviene compararlos con otras dos arquitecturas muy usadas, cada una adaptada a un tipo de dato y de tarea.

**Redes convolucionales (CNN)**: Aplican filtros pequeños que se desplazan sobre la entrada y comparten sus pesos en todas las posiciones. Así detectan patrones locales (bordes y texturas) y, al apilar capas convolucionales y de pooling, se construye una jerarquía de características que va de lo simple a lo complejo. Estas redes se adaptan a tareas de visión: diagnóstico por imagen, clasificación y detección de objetos. También pueden aplicarse a secuencias mediante convoluciones unidimensionales.

**Redes generativas adversarias (GAN)**: Enfrentan a dos redes: un generador G, que produce muestras a partir de ruido z, y un discriminador D, que intenta distinguir las muestras reales de las generadas. Se entrenan como un juego de suma cero. Asimismo, se adaptan a tareas de generación de imágenes, aumento de datos y traducción entre dominios visuales. Sus limitaciones principales son el entrenamiento inestable, el colapso de modos (el generador produce pocas variedades de muestras) y la dificultad para evaluar objetivamente la calidad de lo generado.

Los Transformers también se han extendido más allá del texto: el Vision Transformer divide una imagen en parches y los procesa como una secuencia de tokens, con resultados competitivos cuando se compara a las CNN.

***Tabla Comparativa - Arquitecturas comunes de aprendizaje profundo y su adaptación a tareas específicas:***

| Arquitectura | Dato Típico | Idea Central | Adaptacion a Tareas | Ejemplos de Uso |
|---|---|---|---|---|
| CNN | Imágenes, señales | Filtros con pesos compartidos. Patrones locales y jerarquía de características | Clasificación, detección y segmentación. CNN 1D para señales. | Reconocimiento de objetos, diagnóstico por imágen. |
| RNN / LSTM / GRU | Secuencias, series de tiempo | Estado oculto que se actualiza paso a paso. Compuertas para memoria. | Pronóstico, modelado de lenguaje, voz. Entrada y salida de longitud variable. | Predicción financiera, reconocimiento de voz. |
| GAN | Imágenes, audio | Generador y discriminador que compiten | Generación y aumento de datos. Traducción entre dominios. | Síntesis de imágenes, datos sintéticos. |
| Transformer | Texto, imágenes (parches), audio | Atención entre todas las posiciones. Procesamiento paralelo. | Encoder (comprensión), decoder (generación) o ambos (traducción, resumen). | Traducción, resumen, generación de texto, visión. |

## 3) Análisis crítico de aplicaciones reales

***Caso 1 (GRU-LSTM): reconocimiento de voz en cinco idiomas de la India***

**Problema abordado**

En Gupta et al. (2026) se aborda el reconocimiento de voz en entornos ruidosos para cinco idiomas de la India: bangla, malayalam, odia, telugu y urdu. El análisis tiene sus dificultades debido a que la señal de voz cambia en el tiempo según el hablante, el acento y el entorno. Los autores usan 8,682 muestras de audio de repositorios de Kaggle: 1,527 en bangla, 2,393 en malayalam, 747 en odia, 1,515 en telugu y 2,500 en urdu. Con esas muestras, se extraen características con la librería Librosa usando espectrograma, STFT, spectral rolloff, ancho de banda espectral, MFCC y cromagrama. Me gustaría recalcar que el artículo se trata de un preprint publicado en Research Square el 18 de mayo del 2026, debido a que es un preprint todavía no ha pasado por revisión por pares.

**Arquitectura y justificación**

Proponen un modelo híbrido apilado: una capa LSTM de 128 unidades que devuelve toda la secuencia, seguida de una capa de GRU de 64 unidades, una capa densa de 64 neuronas con activación ReLU, dropout de 0.3 y una capa densa final de 5 neuronas con softmax. Se entrena con el optimizador Adam durante 50 épocas con lotes de 32 y la tasa de aprendizaje y el dropout se seleccionan manualmente probando tres valores de cada uno. La mejor configuración es con una tasa de aprendizaje de 0.001 con dropout de 0.3. El modelo híbrido se justifica, porque el audio es una secuencia con dependencias temporales. La LSTM conserva información de largo plazo con una celda y sus tres compuertas. La GRU aporta una unidad más ligera, porque tiene dos compuertas. Al combinar ambas variantes de RNN, trabajan en conjunto. El modelo se compara con una RNN simple, una GRU, una LSTM y una combinación RNN-GRU simple.

**Beneficios**

- La mejor configuración alcanza 97.02% de exactitud de validación según el resumen y la conclusión (la Tabla 2 del artículo muestra 97.04%). Por idioma obtiene 96.56% en bangla, 96.56% en malayalam, 98.56% en odia, 94.59% en telugu y 98.95% en urdu.
- Comparando la RNN simple con los modelos con compuertas (LSTM, GRU e híbrido), se alcanza entre 90.69% y 99%. Además demuestran una curva más estable. De esta forma, se demuestra la ventaja de las compuertas sobre la RNN simple. En el caso de la RNN simple, la exactitud de validación oscilaba entre 64.59% y 72.59% con curvas inestables.
- El modelo es accesible en recursos, porque el artículo no menciona GPU y los procesadores utilizados son Intel Core i5 y 8 GB de RAM.

**Limitaciones reportadas**

Los autores indican que el modelo puede requerir más tiempo de entrenamiento si los datos son grandes y complejos. También mencionan que un entrenamiento prolongado tiene el riego

de generar un sobreajuste en el modelo. Adicionalmente, los autores mencionan que posibles mejoras futuras son entrenarlo con otros idiomas y aplicar técnicas de reducción de ruido.

**Valoración crítica**

Este artículo es útil, porque coincide con la aplicación sugerida de reconocimiento de voz y demuestra la ventaja de las compuertas sobre una RNN simple. Sin embargo, al revisar el texto completo encontré varios puntos que no me dejan satisfecho con las conclusiones del artículo:

- Estado de trabajo: Es un preprint sin revisión por pares, debido a esto los resultados no deben tratarse como validados.
- Inconsistencias internas: El resumen y la conclusión indican tasas de aprendizaje de 0.1, 0.001 y 0.0001, pero la Tabla 2 y el texto de la sección 4.2 usan 0.01. La mejor exactitud de validación aparece como 97.02% y como 97.04%. El texto da 94.56% para telugu mientras que la Tabla 3 muestra 94.59%. Además, la pérdida se rotula como porcentaje, pero mezcla escalas, por ejemplo 0.06 y 11.78 en la misma columna de la de la Tabla 2.
- Métricas incoherentes: El F1 se define como la media armónica de precisión y recall en la ecuación 13 del artículo, pero varios valores no la cumplen. Por ejemplo, para bangla la presión es de 98.5 y el recall es de 97.5 lo cual implica un F1 cercano a 98.0, pero en el artículo indica un valor de 95. En el caso de urdu con valores de 99 en ambas métricas de precisión y recall, el F1 debería de ser 99, no 96 como viene reportado. Finalmente, en telugu la precisión de 56 y recall de 65 con la RNN no son compatibles con un F1 de 96.
- La conclusión no está respaldada por la evidencia: El artículo afirma que el modelo logra la mayor exactitud con la menor pérdida en los cinco idiomas, pero su propia Tabla 4 muestra que en urdu la GRU obtiene 99% de exactitud de validación comparado a 98.85% del modelo híbrido. Además, en malayalam la LSTM tiene una pérdida de validación de 0.2310 comparado a 0.599 del híbrido. En el caso de telugu, la diferencia con la GRU es de solamente 0.06 puntos, 94.59% comparado a 94.53%. Finalmente, estas son diferencias pequeñas medidas en una ejecución, sin repeticiones ni intervalos de confianza.
- Tareas y datos: La capa final tiene 5 neuronas con softmax, lo que quiere decir que el modelo clasifica cada muestra en uno de cinco idiomas (clasificación de idioma) y no transcribe voz a texto.
- Ruido: el título destaca los entornos ruidosos, pero en el texto revisado no hay una descripción de cómo se introdujo o midió el ruido a los datos. Tampoco se discuten resultados por nivel de ruido.
- Partición y selección: Se emplea una partición holdout sin proporción especificada.

En resumen, el caso ilustra bien el papel de las compuertas y ofrece material para un análisis crítico, pero en un informe académico debe citarse como preprint y complementarse con fuentes que sí sean revisadas por pares.

***Caso 2 (Transfomer): Vision Transformers y la autoatención como columna vertebral***

**Problema abordado**

En Waseem et al. (2026) se presenta una revisión sistemática (survey) de los Vision Transformers (ViT) como columna vertebral (backbone) para la clasificación de imágenes. El problema es que las CNN usan filtros de tamaño fijo que capturan contexto local y tienen dificultad para modelar dependencias globales. A comparación, la autoatención de los Transformers sí tiene la capacidad de modelar las dependencias globales. Una de las desventajas es que se usan muchos parámetros, tienen un alto costo computacional y tienen una complejidad cuadrática respecto al tamaño de la entrada. Estas desventajas limitan su escalabilidad y su despliegue. Por estos motivos, la revisión organiza las propuestas que mejoran la autoatención o la combinan con otros bloques. Además, cubre su transferencia a modelos de lenguaje multimodales y generativos que son alternativas basadas en modelos de espacio de estados (SSM) y los métodos de explicabilidad. Este es un artículo de acceso abierto, recibido en septiembre del 2025 y aceptado en abril del 2026.

**Arquitectura y justificación**

La arquitectura de referencia es el ViT original (Dosovitskiy et al., 2021): la imagen se divide en parches no solapados de tamaño fijo que se aplanan y proyectan linealmente a vectores de embedding. Después, se le suman codificaciones posicionales y un token de clase. Luego, se procesan con un encoder Transformer estándar formado por autoatención multi-cabeza, normalización y una red feed-forward (MLP). Cada parche cumple el papel de un token y se proyecta a Query, Key y Value con la misma ecuación de atención de la sección 2.5. La justificación es que la autoatención modela dependencias globales sin los fuertes sesgos inductivos de las CNN y que con preentrenamiento a gran escala sirve como representación transferible a muchas tareas. A partir de ahí, la encuesta propone una taxonomía de variantes que modifican el diseño de la atención: fusión con convolución, atención jerárquica, atención cruzada, fusión de atenciones, atención por ventanas y axial y manipulación de tokens.

**Beneficios**

- Según la revisión, los ViT superan a las CNN de referencia en varias métricas y se han vuelto una alternativa habitual en casi todas las tareas de visión.
- En lugar de depender solamente de mayor cantidad de parámetros, los modelos de mejor desempeño reducen el costo combinado de atención jerárquica, atención localizada y agregación global selectiva. Por ejemplo: en ImageNet-1k, DualViT alcanza 86.5% de exactitud Top-1 con 75 millones de parámetros y 60.5 GFLOPs a 384 x 384, mientras que DaViT llega a 90.4% de exactitud con 1,437 millones de parámetros y 1,038 GFLOPs a 512 x 512.
- El mecanismo se transfiere a modelos de lenguaje multimodales y marcos generativos, lo que lo convierte en una base común para varias tareas.

**Limitaciones**

El artículo identifica como desafíos los siguientes puntos:

- La complejidad cuadrática de la autoatención respecto al número de parches, agravada con imágenes de alta resolución o secuencias largas.
- El gran número de parámetros y el alto presupuesto computacional.
- Cabezas de atención que aprenden representaciones redundantes y colapso de la atención al aumentar la profundidad.
- La dificultad para generalizar entre conjuntos de datos, resoluciones y cambios de dominio, junto con la dependencia de aumento de datos, ajuste arquitectónico y preentrenamiento a gran escala.
- La falta de un diseño estándar para alternativas como los modelos de espacio de estados que no ofrecen interacciones completas entre pares de posiciones como la autoatención.

**Valoración crítica**

Por ser una revisión y no un experimento, su aporte es de síntesis debido a que no entrena modelos propios y toma las cifras de los artículos originales. Por ende, los modelos comparados pueden diferir en configuraciones de entrenamiento y resolución. Además, seleccionar los diez mejores por grupo de tamaño le da favoritismo a los de mayor exactitud. Esto limita las conclusiones sobre eficiencia, porque la comparación entre DualViT y DaViT mezcla resoluciones distintas. La revisión también se centra en la clasificación de imágenes y remite a otras revisiones para el resto de tareas. A pesar de todas sus limitaciones, es una fuente actual, revisada por pares y de acceso abierto que permite analizar los beneficios y los límites de los Transformers.

Esta revisión está conectada a mi implementación práctica: la complejidad cuadrática de la atención está relacionada con el límite de 512 tokens con que se truncan los artículos en T5. Asimismo, la dependencia del preentrenamiento a gran escala se refleja en el aspecto que T5-small ya funciona sin entrenamiento adicional y mejora poco con un ajuste fino corto. Como limitación del enfoque, una revisión no es una implementación concreta, por eso se toma el ViT como la aplicación analizada y la revisión como la fuente que lo documenta.

***Comparación de ambos casos***

El impacto de estas de estas arquitecturas puede resumirse como un tránsito de sistemas con componentes diseñados a mano a modelos entrenados de extremo a extremo. Los Transformers aportaron escalabilidad, porque utilizan GPUs y preentrenamiento masivo. En contraste, LSTM y GRU conservan su lugar donde los datos son escasos, los recursos limitados o la señal es una serie numérica.

| Aspecto | Caso 1: GRU-LSTM (Gupta et al., 2026, preprint) | Caso 2: Vision Transformers (Waseem et al., 2026) |
|---|---|---|
| Dominio / tarea | Voz: clasificación de muestras de audio en cinco idiomas de la India | Visión por computadora: clasificación de imágenes (revisión de la literatura). |
| Tipo de fuente | Preprint sin revisión por pares. Experimento propio de los autores. | Artículo de revisión revisado por pares. Sin experimentos propios. |
| Motivo de la arquitectura | Dependencias temporales: LSTM de memoria larga más GRU más ligera. | Autoatención modela dependencias globales sin el sesgo local de las CNN. Transferible con preentrenamiento. |
| Fortaleza principal | Supera con amplio margen a la RNN simple. Se usa hardware modesto. | Supera a las CNN en varias métricas. Es la base común para tareas multimodales y generativas. |
| Limitación principal | Inconsistencias entre tablas y conclusiones. Ruido no descrito. Una sola ejecución. | Es de costo cuadrático y tiene muchos parámetros. Dependencia de preentrenamiento. Cifras comparadas tomadas de otros artículos. |
| Riesgo práctico | Sobreestimar el desempeño por la partición y la selección de hiperparámetros. | Ineficiencia con alta resolución y dificultad de despliegue en dispositivos limitados. |

## 4) Implementación comparado en código de dos modelos CNN:

Ambas implementaciones se encuentran en el Google Colab con el código comentado y los resultados:
https://colab.research.google.com/drive/1g61Qw8hEDZ_Wuov0HXFHvm-zhORvAmnh?usp=sharing

***Implementación 1: LSTM para generación de texto***

**Preparación de datos**

Se utilizó el corpus *Tiny Shakespeare* de 1,115,394 caracteres. El texto se modeló a nivel de carácter: se construyó un vocabulario con los 65 caracteres únicos, se codificó cada carácter a un entero y se dividió el corpus en 90% para entrenamiento (1,003,854 caracteres) y 10% para validación (111,540 caracteres) respetando el orden del texto, sin mezclarlo, para evitar que ventanas traslapadas aparezcan en ambos conjuntos. Durante el entrenamiento se muestrean ventanas aleatorias de 100 caracteres, donde el objetivo es la misma ventana desplazada un carácter, de modo que el modelo aprende a predecir el siguiente carácter.

**Arquitectura y configuración del modelo LSTM**

| Componente | Configuración |
|---|---|
| Capa de entrada | Embedding de 128 dimensiones sobre el vocabulario de caracteres |
| Capas recurrentes | LSTM de 2 capas, 256 unidades ocultas, dropout 0.3 entre capas |
| Capa de salida | Dropout 0.3 + capa lineal hacia el vocabulario (softmax implícito en la pérdida) |
| Pérdida y optimizador | Entropía cruzada. Adam, tasa de aprendizaje $2 \times 10^{-3}$ |
| Regularización / estabilidad | Dropout y recorte de gradiente (norma máxima 1,0) |
| Entrenamiento | 3,000 pases, lotes de 128 ventanas de 100 caracteres, valor de semilla es 42 |
| Parámetros totales | 946,625 |

**Flujo de datos y formas de los tensores en el modelo LSTM**

La siguiente tabla sigue un lote de entrenamiento (128 ventanas de 100 caracteres) a lo largo del modelo e indica la forma del tensor tras cada capa y el papel que cumple:

| Etapa | Forma de salida | Capa | Qué hace |
|---|---|---|---|
| Entrada x | (128, 100) | Enteros | Cada número identifica a un carácter del vocabulario (65 caracteres distintos). |
| Embedding | (128, 100, 128) | nn.Embedding | Busca el vector de 128 valores de cada carácter en una tabla aprendida. Todas las apariciones del mismo carácter reciben el mismo vector. |
| LSTM - capa 1 | (128, 100, 256) | nn.LSTM | Lee la secuencia paso a paso. Con sus compuertas actualiza una memoria de 256 valores y produce un estado por posición que ya depende del contexto previo. |
| LSTM - capa 2 | (128, 100, 256) | nn.LSTM | Procesa los estados de la capa 1. Entre ambas capas se aplica dropout de 0.3. |
| Dropout | (128, 100, 256) | nn.Dropout | Apaga al azar el 30% de los valores durante el entrenamiento para reducir el sobreajuste. No actúa al evaluar ni generar. |
| Capa lineal | (128, 100, 65) | nn.Linear | Convierte los 256 valores en 65 puntuaciones (logits). Una puntuación por cada carácter posible. |

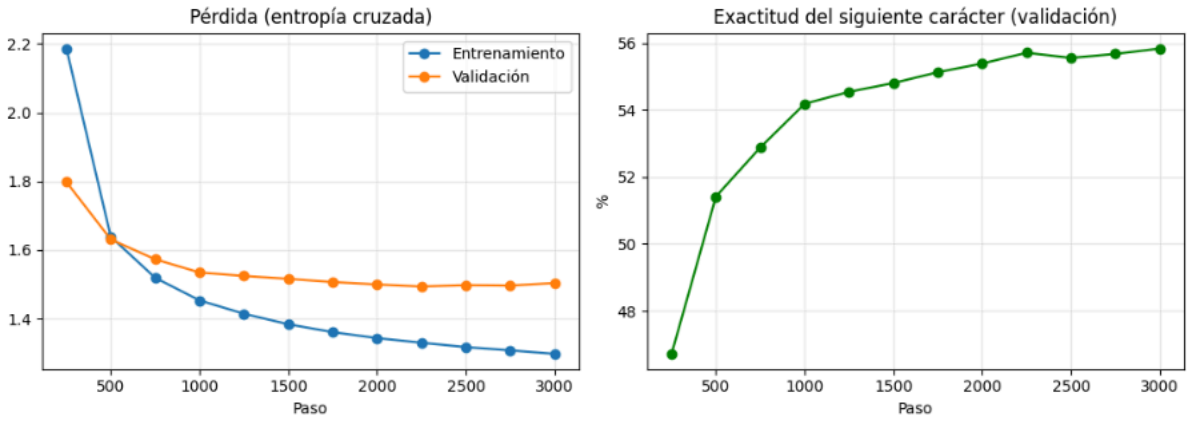
| | | | |
|---|---|---|---|
| Pérdida | Un escalar | Entropía cruzada | Compara las puntuaciones con el carácter correcto y la ventana desplazada a un lugar. Mide el error que se propaga hacia atrás. |

**Resultados de LSTM**

| Métrica (conjunto de validación) | Valor |
|---|---|
| Pérdida de entropía cruzada | 1.504 |
| Perplejidad (una predicción aleatoria daría 65) | 4.50 |
| Exactitud del siguiente carácter | 55.75% |
| Tiempo de entrenamiento (3,000 pasos) | 113.2 s |
| Distinct-2 (T = 0.5 / 0.8 / 1.2) | 0.953 / 0.990 / 1.000 |

Las métricas finales corresponden a una evaluación con 50 lotes de validación. El registro de entrenamiento (cada 250 pasos) muestra un último valor de 55.84% de exactitud y pérdida de 1.503, ligeramente distinto por el muestreo aleatorio de los lotes.

**Curvas de aprendizaje del LSTM. Izquierda: pérdida de entropía cruzada en entrenamiento y validación. Derecha: exactitud del siguiente carácter en validación.**




La siguiente tabla muestra fragmentos del texto generado a partir del prompt “ROMEO” con tres temperaturas. La temperatura controla la aleatoriedad del muestreo: valores bajos favorecen los caracteres más probables y valores altos aumentan la variedad a costa de más errores.

| Fuente | Texto |
|---|---|
| T = 0.5 | ROMEO: I cannot noble party of the death; And yet you shall be made a base parts be well, And here we will confess and comfort of the wind, And hear me weep and have heard my hands shall be so: [...] |
| T = 0.8 | ROMEO: I cannot noble party of my mother; Where is the unherence of his slapping brave. DUKE VINCENTIO: Go, by the devil that conceive me post him, That thou liest great father; [...] QUEEN ELIZABETH: To send me storm'd than you have been be last one. [...] |
| T = 1.2 | ROMEO: It in thence this sea-time modesty. The bury she use them, and be lapping by well, or the state hath not indeed to fall conceits; O, storous amparted urged themselves: have Henry Northutwive wurn him as your powers; [...] |

**Interpretación**

El modelo aprendió estructura del lenguaje de forma clara. La perplejidad de validación bajó de 6.04 (paso 250) a 4.50 (paso 3,000), comparado a 65 que obtendría una predicción aleatoria sobre 65 caracteres y la exactitud del siguiente carácter alcanzó 55.75%. La mayor parte de la mejora ocurrió en los primeros 1,000 pasos. Desde el paso 2,250 la pérdida de validación se estancó en torno a 1.49-1.50 mientras siguió bajando hasta 1.297, lo que indica el inicio de un sobreajuste. El entrenamiento completo tomó 113.2s.

En la generación con T = 0.5, el texto usa palabras reales y versos plausibles, pero sin coherencia de significado y con conectores repetidos. En el caso de T = 0.8, se reproduce el formato de una obra de teatro (nombres de personajes en mayúsculas seguidos de su diálogo) con pocas palabras inventadas. Finalmente con T = 1.2, aparecen numerosas palabras inexistentes (“storous” o “amparted”), aunque se mantiene la forma de los versos. La diversidad medida con distinct-2 sube con la temperatura (0.953, 0.990 y 1.000), pero, desafortunadamente, esta métrica mide variedad y no calidad. Con solamente 500 caracteres casi cualquier bigrama es único y con T - 1.2, los bigramas con palabras inventadas también cuentan como únicos. En conjunto, un LSTM a nivel de carácter aprende rápido la estructura superficial como ortografía, versos y formato, pero no la coherencia semántica de largo plazo.

***Implementación 2: Transformer preentrenado (T5-small) para resumen***

**Preparación de datos**

Se usó el conjunto *CNN/DailyMail 3.0.0* cargado con la librería datasets. Para la evaluación se tomaron los primeros 100 artículos del conjunto de prueba (0.87% de los 11,490 disponibles). Cada entrada se prefija con "*summarize:*" por convención de T5. Después, la entrada se tokeniza con el tokenizador del modelo y se trunca a 512 tokens. Las referencias son los *highlights* escritos por humanos. Para el ajuste fino se muestrearon al azar 2,000 ejemplos de los 287,113 del conjunto de entrenamiento (0.70%) con los tokens de relleno enmascarados en la pérdida.

**Arquitectura y configuración del modelo T5-small**

| Componente | Configuración |
|---|---|
| Modelo | google-t5 / t5-small (Transformer encoder-decoder preentrenado) |
| Librería | Hugging Face Transformers: AutoTokenizer y AutoModelForSeq2SeqLM |
| Generación | Beam search con 4 haces. Máximo de 80 tokens nuevos. no_repeat_ngram_size = 3 |
| Ajuste fino | AdamW, tasa de aprendizaje $3x10^{-4}$, lote 8, 1 época sobre 2,000 ejemplos, recorte de gradiente 1.0 |
| Métrica | ROUGE-1, ROUGE02 y ROUGE-L con la librería evaluate |
| Parámetros totales | 60,506,624 (unas 64 veces los del LSTM) |

**Flujo de datos a través de T5-small**

La siguiente tabla resume el camino de un artículo a través de T5-small. T5-small es un Transformer encoder-decoder. El encoder y el decoder tienen 6 bloques apilados cada uno.

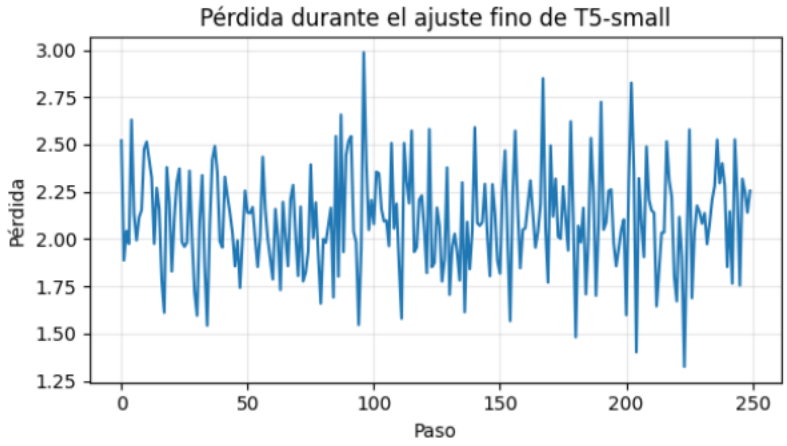
| Etapa | Forma (lote, tokens, dim.) | Qué hace |
|---|---|---|
| Tokenización | (lote, 512) | El texto con el prefijo “summarize:” se divide en subpalabras y se convierte en enteros. Los artículos con más de 512 tokens se truncan y los más cortos se rellenan para que todos los de un lote midan lo mismo. |
| Embedding | (lote, 512, 512) | Cada token se convierte en un vector. T5 codifica la posición de forma relativa dentro de la atención. |
| Encoder (6 bloques) | (lote, 512, 512) | Cada bloque aplica auto-atención multi-cabeza y una red feed-forward con conexiones residuales y normalización. Produce una representación contextual del artículo. |
| Decoder ( 6 bloques) | (lote, tokens generados, 512) | Auto atención enmascarada sobre lo ya generado, atención cruzada hacia la salida del encoder y red feed-forward. |
| Capa de salida | (lote, tokens generados, vocabulario) | Proyecta a una puntuación por cada token del vocabulario. La búsqueda por haz elige la secuencia final. |
| Decodificación | Texto | Los identificadores generados se convierten de nuevo en el resumen legible. |

**Resultados de T5-small en CNN/DailyMail**

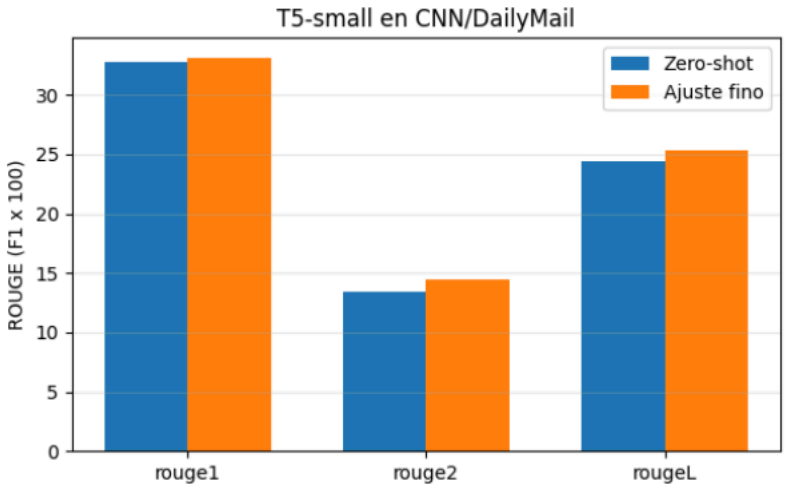
| Condición | ROUGE-1 | ROUGE-2 | ROUGE-L |
|---|---|---|---|
| Zero-shot (sin entrenamiento adicional) | 32.76 | 13.44 | 24.47 |
| Tras ajuste fino corto | 33.14 | 14.41 | 25.29 |

El ajuste fino (250 pasos, 1 época sobre 2,000 ejemplos) tomó 96.7 s. La inferencia sobre los 100 artículos de prueba tomó 16.6 s con el modelo original (0.17 s por artículo) y unos 15.9 s con el modelo ajusto (0.159 s por artículo). La diferencia es pequeña y no indica una mejora real de velocidad.

**La siguiente gráfica muestra la pérdida por lote durante el ajuste fino de T5-small (250 pasos con lotes de 8 ejemplos):**



**Esta gráfica muestra ROUGE de T5-small en CNN/DailyMaill. Zero-shot comparado a ajuste fino (100 artículos):**




**La siguiente tabla presenta ejemplos generados. Son los resúmenes de referencia, zero-shot y con ajuste fino de dos artículos del conjunto de pruebas.**

| Fuente | Texto |
|---|---|
| Ej. 1: referencia | Membership gives the ICC jurisdiction over alleged crimes committed in Palestinian territories since last June. Israel and the United States opposed the move, which could open the door to war crimes investigations against Israelis. |
| Ej. 1: zero-shot | the palestinians signed the ICC's founding Rome Statute in January. it also accepted its jurisdiction over alleged crimes committed in the occupied territories. a preliminary examination into the situation in Palestinian territories was opened. |
| Ej. 1: ajuste fino | The Palestinian Authority officially became the 123rd member of the International Criminal Court on Wednesday. The ICC opened a preliminary examination into the situation in Palestinian territories. Israel and the United States opposed the Palestinians' efforts to join the body. |
| Ej. 2: referencia | Theia, a bully breed mix, was apparently hit by a car, whacked with a hammer and buried in a field. “She's a true miracle dog and she |

| | |
|---|---|
| | deserves a good life," says Sara Mellado, who is looking for a home for Theia. |
| Ej. 2: zero-shot | theia is a friendly white-and-black bully breed mix. she was found by a worker who took her to a vet for help. the dog's brush with death did not leave her unscathed. |
| Ej. 2: ajuste fino | A stray pooch in Washington State has used up at least three of her own after being hit by a car. Theia, a friendly white-and-black bully breed mix, has been receiving care at the Veterinary Teaching Hospital. Four days after her apparent death, the dog managed to stagger to a nearby farm, dirt-covered and |

**Interpretación**

El ajuste fino mejoró ligeramente las métricas: +0.38 puntos en ROUGE-1, +0.97 en ROUGE-2 y +0.82 en ROUGE-L. Con solamente 100 artículos de evaluacion, diferencias menores a un punto no pueden considerarse concluyentes, porque no se calculó un intervalo de confianza y la pérdida del ajuste fino oscila entre 1.3 y 3.0 sin una tendencia descendente clara por lo que la mejora observada es modesta. Esto es coherente con que T5-small ya parte de un preentrenamiento multitarea que incluye resumen. De tal forma que el escenario zero-shot no corresponde a una tarea totalmente desconocida para el modelo.

En los ejemplos de la tabla anterior, ambos modelos generan resúmenes mayormente extractivos, copian oraciones del artículo en un lugar de condensarlo. El modelo zero-shot selecciona oraciones intermedias y omite hechos centrales de la referencia. Con el ajuste fino, los resúmenes comienzan con la primera oración del artículo, lo cual es un comportamiento consistente con el sesgo hacia el inicio frecuente en noticias y capturan algo más de la información principal, aunque ningún ejemplo cubre todos los puntos del resumen de referencia. En el ejemplo 2, se puede apreciar que el límite de 80 tokens nuevos truncó un resumen a mitad de la frase. Un punto adicional es que ROUGE mide solapamiento de palabras con la referencia y no la fidelidad factual. Básicamente, es un resumen correcto pero parafraseado puede tener una puntuación baja y uno con datos erróneos puede tener una puntuación alta.

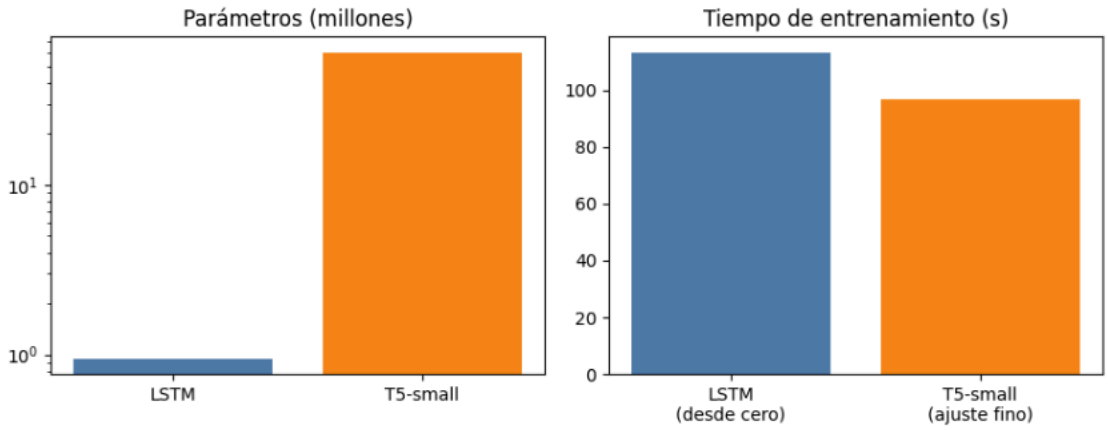
**Comparación entre ambos enfoques**

Los objetivos son distintos: generación libre comparado a resumen condicionado. Por ende, las métricas principales no son directamente comparables. Para poder hacer una buena comparación, se tiene que comparar dimensiones comunes.

**Comparación de rendimiento, complejidad y aplicabilidad**

| Dimension | LSTM (desde cero) | T5-small (preentrenado) |
|---|---|---|
| Rendimiento | Perplejidad 4.50 (azar: 65) y exactitud 55.75%. Texto con formato de obra teatral y ortografia mayormente correcta, pero sin coherencia global. | Antes del ajuste fino: ROUGE-1 = 32.76, ROUGE-2 = 13.44 y ROUGE-L = 24.47. Después del ajuste fino: ROUGE-1 = 33.14, ROUGE-2 = 14.41 y ROUGE-L = 25.29. Los resúmenes son legibles, pero en los ejemplos revisados copian oraciones del artículo en lugar de reformularlas. |
| Complejidad | 946,625 parámetros. Código y arquitectura simples. Fácil de depurar. | 60,506,624 parámetros (aproximadamente 64 veces más parámetros). Mayor complejidad, pero la librería abstrae la arquitectura. |
| Costo de entrenamiento | Entrenamiento completo desde cero: 113.2 s. | Ajuste corto: 96.7 s (sin contar el preentrenamiento ya realizado por terceros). Inferencia de 0.159 s por artículo. |
| Datos necesarios | Un corpus del dominio. Aprende solamente lo que ve. | Pocos datos o ninguno gracias al transfer learning. |
| Dependencias largas | Limitados por el estado oculto y el procesamiento secuencial. | Atención directa entre posiciones. Límite de 512 tokens y costo cuadrático. |
| Paralelismo | Bajo en el tiempo (pasos secuenciales). | Alto durante el entrenamiento. |
| Aplicabilidad | Series de tiempo, dispositivos con pocos recursos, prototipos rápidos. | Tareas de lenguaje con textos largos y exigencia de calidad. Requiere GPU. |

**Comparación de parámetros (escala logarítmica) y tiempo de entrenamiento entre el LSTM y T5-small:**




Los tiempos de entrenamiento son parecidos (113.2 s comparado a 96.7 s), pero no son comparables en esfuerzo total: el LSTM aprende todo desde cero, mientras que T5-small aprovecha un preentrenamiento masivo hecho por terceros. En el caso de T5-small y el transfer learning, aunque tenga 64 veces más parámetros, necesita solamente un ajuste corto.

## 5) Conclusión:

***Mapa Conceptual***

Las compuertas de LSTM y GRU resolvieron en gran medida el desvanecimiento del gradiente de las RNN. Además, el recorte de gradiente controla la explosion de gradiente. Finalmente, la atención llevó la idea más lejos al conectar directamente cualquier par de posiciones y permitir el cómputo en paralelo, a costa de más consumo de memoria.

***Casos reales***

El caso GRU-LSTM (Gupta et al., 2026) muestra que las redes recurrentes con compuertas superan con claridad a una RNN simple en señales de audio, pero también que un resultado publicado debe leerse con cautela cuando es un preprint con inconsistencias entre tablas y conclusiones. En el caso de los Vision Transformers (Waseem et al., 2026) muestra que la autoatención es una base potente y transferible, con el costo cuadrático y la dependencia de preentrenamiento a gran escala como principales limitaciones.

***Código LSTM***

El LSTM a nivel de carácter alcanzó una perplejidad de validación de 4.50 y una exactitud de 55.75% en 113.2 s de entrenamiento al aprender ortografía y formato de diálogo, pero no coherencia semántica. Como se puede apreciar en las gráficas, la validación se estancó desde el paso 2250, mientras que el entrenamiento seguía mejorando y la temperatura mostró el compromiso entre diversidad y calidad.

***Código Transformer***

T5-small obtuvo ROUGE-1 de 32.76 sin entrenamiento adicional y de 33.14 después de un ajuste fino de 96.7 s. La mejora es modesta y no concluyente con 100 artículos y los resúmenes son mayormente extractivos.

***Comparación y recomendación***

Con tiempos de entrenamiento similares, T5-small ofrece resultados útiles desde el inicio gracias al preentrenamiento, a pesar de que tiene 64 veces más parámetros. Personalmente, recomendaría un Transformer preentrenado para tareas de lenguaje como textos largos. En contraste, un LSTM o GRU son útiles para series numéricas, prototipos rápidos o entornos con pocos recursos.

***Limitaciones del estudio***

Siendo realista, no se puede medir la fidelidad factual con una sola ejecución con una semilla, evaluación sobre 100 artículos, salida limitada a 80 tokens y uso de ROUGE. Se tienen que hacer variaciones a los artículos, cantidad de tokens y semillas para tener resultados concluyentes.

***Arquitecturas***

Cada arquitectura se adapta a un tipo de dato: las CNN a patrones espaciales, las RNN a secuencias, las GAN a la generación mediante competencia y los Transformers a contextos amplios con procesamiento paralelo. La elección depende del tipo de dato, el volumen de información y los recursos disponibles.

***Mejoras propuestas***

a) **LSTM**: entrenar más pasos, usar un modelo a nivel de subpalabra y compara con GRU.
b) **T5**: probar T5-base o BART, aumentar el ajuste fino y evaluar con métricas semánticas (BERTScore) o hacer una revisión humana para validar la fidelidad factual.
c) **Casos reales**: contrastar los resultados del preprint con una versión revisada por pares, repetir los experimentos con varias semillas e intervalos de confianza y preferir variantes de atención eficientes (por ventanas o jerárquicas) cuando el cómputo sea limitado.

***Aplicaciones innovadoras***

Modelos que usen LSTM y GRU para señales locales en edge devices, Transformers para el análisis profundo en computación en la nube y herramientas de resumen asistido con verificación humana en industrias de suma importancia como en el área médica tomando en cuenta que ROUGE no mide la fidelidad factual.

## 6) Referencias:


Gupta, A., Kumar, R., Gupta, M., Srivastava, D., & Garg, A. (2026). *A novel hybrid GRU-LSTM model for robust speech recognition of Indian languages in noisy environments* [Preprint]. Research Square. https://doi.org/10.21203/rs.3.rs-9454210/v1

Waseem, A., Ruiu, P., Nixon, S., Lagorio, A., & Tistarelli, M. (2026). Self-attention as the backbone: A survey on Vision Transformers. *Computer Vision and Image Understanding, 269*, Article 104791. https://doi.org/10.1016/j.cviu.2026.104791